1 Смежные классы и классы эквивалентности

Рассмотрим линейный двоичный код

$$\mathcal{C} = \{\mathbf{c}_1, \mathbf{c}_2, \ldots, \mathbf{c}_M\} \subset \mathbb{F}_2^n,$$

где $M = 2^k$. Векторное пространство $\mathbb{F}_2^n$ содержит 2^n элементов, а код $\mathcal{C}$ представляет собой подмножество из 2^k кодовых слов.

Для произвольного вектора $\mathbf{a} \in \mathbb{F}_2^n$ определим сдвиг кода:

$$\mathcal{C} + \mathbf{a} = \{\mathbf{c}_1 + \mathbf{a}, \mathbf{c}_2 + \mathbf{a}, \ldots, \mathbf{c}_M + \mathbf{a}\}.$$

Возможны две ситуации:

- Если $\mathbf{a} \in \mathcal{C}$, то $\mathcal{C} + \mathbf{a} = \mathcal{C}$, поскольку сумма любых двух кодовых слов также принадлежит коду.
- Если $\mathbf{a} \notin \mathcal{C}$, то $\mathcal{C} + \mathbf{a}$ образует новый смежный класс, отличный от исходного кода.

Два смежных класса $\mathcal{C} + \mathbf{a}$ и $\mathcal{C} + \mathbf{b}$ совпадают тогда и только тогда, когда выполняется условие

$$\mathbf{a} + \mathbf{b} \in \mathcal{C}.$$

Действительно, если существуют $\mathbf{c}_i, \mathbf{c}_j \in \mathcal{C}$ такие, что

$$\mathbf{c}_i + \mathbf{a} = \mathbf{c}_j + \mathbf{b},$$

то

$$\mathbf{a} + \mathbf{b} = \mathbf{c}_i + \mathbf{c}_j \in \mathcal{C}.$$

Обратное утверждение проверяется аналогично: из $\mathbf{a} + \mathbf{b} \in \mathcal{C}$ следует совпадение сдвигов.

Таким образом, на множестве $\mathbb{F}_2^n$ определено отношение эквивалентности

$$\mathbf{a} \sim \mathbf{b} \iff \mathbf{a} + \mathbf{b} \in \mathcal{C},$$

а класс эквивалентности вектора $\mathbf{a}$ имеет вид

$$[\mathbf{a}] = \mathcal{C} + \mathbf{a}.$$

Отсюда вытекают важные следствия:

- Все векторы пространства $\mathbb{F}_2^n$ разбиваются на непересекающиеся смежные классы.
- Каждый смежный класс содержит ровно 2^k элементов.
- Количество смежных классов равно
$$\frac{2^n}{2^k} = 2^{n-k}.$$

Именно поэтому для линейного (n, k)-кода возникает 2^{n-k} различных классов ошибок, а синдром естественным образом нумерует эти классы.

2 Декодирование по максимуму правдоподобия

Рассмотрим дискретный канал без памяти, заданный переходными вероятностями
$$P(y \mid x),$$
где $x \in \mathcal{X}$ — символ на входе, $y \in \mathcal{Y}$ — символ на выходе.

Пусть передано кодовое слово
$$\mathbf{c}_m = (c_{m1}, c_{m2}, \ldots, c_{mn}),$$
а на приёмной стороне наблюдается вектор
$$\mathbf{y} = (y_1, y_2, \ldots, y_n).$$

Функция правдоподобия для m-го кодового слова определяется как
$$L_m(\mathbf{y}) = P(\mathbf{y} \mid \mathbf{c}_m).$$

Правило декодирования по максимуму правдоподобия формулируется следующим образом:
$$\hat{\mathbf{c}} = \arg\max_{\mathbf{c}_m \in \mathcal{C}} P(\mathbf{y} \mid \mathbf{c}_m).$$

В силу отсутствия памяти у канала, передача отдельных символов независима, поэтому
$$P(\mathbf{y} \mid \mathbf{c}_m) = \prod_{i=1}^{n} P(y_i \mid c_{mi}).$$

Переходя к логарифмической шкале, получаем:

$$\ln P(\mathbf{y} \mid \mathbf{c}_m) = \sum_{i=1}^{n} \ln P(y_i \mid c_{mi}).$$

Вводя локальную метрику

$$\lambda_i(y_i, c_{mi}) = \ln P(y_i \mid c_{mi}),$$

приходим к эквивалентному критерию:

$$\ln P(\mathbf{y} \mid \mathbf{c}_m) = \sum_{i=1}^{n} \lambda_i(y_i, c_{mi}).$$

Таким образом, задача декодирования сводится к поиску кодового слова, максимизирующего сумму локальных метрик.

3 Логарифмическое отношение правдоподобия

Для двоичного кода введём логарифмическое отношение правдоподобия

$$\ell(y_i) = \ln \frac{P(y_i \mid 1)}{P(y_i \mid 0)}.$$

Знак этой величины указывает на предпочтительный символ:

- $\ell(y_i) > 0$ свидетельствует в пользу гипотезы о передаче 1;
- $\ell(y_i) < 0$ свидетельствует в пользу гипотезы о передаче 0;
- $\ell(y_i) = 0$ означает, что наблюдение не даёт преимущества ни одной из гипотез.

Правило принятия решения для отдельного символа принимает вид

$$\ell(y_i) \gtrless 0.$$

Для всего кодового слова удобно составить вектор надёжности

$$\mathbf{L}(\mathbf{y}) = (\ell(y_1), \ell(y_2), \ldots, \ell(y_n)).$$

Введём также двуполярное представление кодового слова:

$$\boldsymbol{\xi}(\mathbf{c}_m) = (2c_{m1} - 1, 2c_{m2} - 1, \ldots, 2c_{mn} - 1),$$

где 0 заменяется на -1, а 1 — на $+1$.

Критерий максимума правдоподобия тогда можно записать как максимизацию скалярного произведения:

$$\hat{\mathbf{c}} = \arg\max_{\mathbf{c}_m \in \mathcal{C}} \langle \boldsymbol{\xi}(\mathbf{c}_m), \mathbf{L}(\mathbf{y}) \rangle.$$

3.1 Обоснование появления скалярного произведения

Рассмотрим два кодовых слова $\mathbf{c}_m$ и $\mathbf{c}_{m'}$. Разобьём множество позиций на четыре категории:

$$I_{ab} = \{i \in \{1, \ldots, n\} : c_{mi} = a,\ c_{m'i} = b\}, \quad a, b \in \{0, 1\}.$$

Множества I_{00} и I_{11} не вносят вклада в разность метрик, поскольку в этих позициях символы совпадают. Значимыми оказываются лишь позиции из I_{10} и I_{01}. После замены 0 на -1 и 1 на $+1$ разность скалярных произведений принимает вид

$$\langle \boldsymbol{\xi}(\mathbf{c}_m), \mathbf{L}(\mathbf{y}) \rangle - \langle \boldsymbol{\xi}(\mathbf{c}_{m'}), \mathbf{L}(\mathbf{y}) \rangle = 2 \sum_{i \in I_{10}} \ell(y_i) - 2 \sum_{i \in I_{01}} \ell(y_i).$$

Следовательно, сравнение правдоподобий эквивалентно сравнению соответствующих скалярных произведений.

4 Двоично-симметричный канал

Для двоично-симметричного канала с вероятностью ошибки $p_0 < \frac{1}{2}$ имеем

$$P(y_i \mid x_i) = \begin{cases} 1 - p_0, & y_i = x_i, \\ p_0, & y_i \neq x_i. \end{cases}$$

Тогда логарифмическое отношение правдоподобия принимает всего два значения:

$$\ell(y_i) = \begin{cases} \sigma, & y_i = 1, \\ -\sigma, & y_i = 0, \end{cases} \quad \sigma = \ln \frac{1 - p_0}{p_0} > 0.$$

Если расстояние Хэмминга между принятым словом $\mathbf{y}$ и кодовым словом $\mathbf{c}_m$ равно $d_H(\mathbf{y}, \mathbf{c}_m)$, то число совпадающих позиций составляет $n - d_H(\mathbf{y}, \mathbf{c}_m)$. Отсюда

$$\langle \boldsymbol{\xi}(\mathbf{c}_m), \mathbf{L}(\mathbf{y}) \rangle = \sigma\big((n - d_H(\mathbf{y}, \mathbf{c}_m)) - d_H(\mathbf{y}, \mathbf{c}_m)\big) = \sigma\big(n - 2d_H(\mathbf{y}, \mathbf{c}_m)\big).$$

Поскольку $\sigma > 0$, максимизация скалярного произведения равносильна минимизации расстояния Хэмминга:

$$\arg\max_{\mathbf{c}_m \in \mathcal{C}} \langle \boldsymbol{\xi}(\mathbf{c}_m), \mathbf{L}(\mathbf{y}) \rangle = \arg\min_{\mathbf{c}_m \in \mathcal{C}} d_H(\mathbf{y}, \mathbf{c}_m).$$

Этот же результат можно получить непосредственно из вероятностной модели:

$$P(\mathbf{y} \mid \mathbf{c}_m) = (1 - p_0)^{n - d_H(\mathbf{y}, \mathbf{c}_m)} p_0^{d_H(\mathbf{y}, \mathbf{c}_m)}.$$

При $p_0 < \frac{1}{2}$ данная величина убывает с ростом $d_H(\mathbf{y}, \mathbf{c}_m)$, следовательно,

$$\arg\max_{\mathbf{c}_m \in \mathcal{C}} P(\mathbf{y} \mid \mathbf{c}_m) = \arg\min_{\mathbf{c}_m \in \mathcal{C}} d_H(\mathbf{y}, \mathbf{c}_m).$$

Таким образом, в двоично-симметричном канале декодирование по максимуму правдоподобия эквивалентно декодированию по минимуму расстояния Хэмминга. Если несколько кодовых слов находятся на одинаковом минимальном расстоянии от принятого, возникает неоднозначность, и в качестве ответа можно выбрать любое из них.

5 Представление кода в виде решётки

Любой блочный код допускает представление в виде специального ориентированного графа, называемого решёткой. Основные свойства такого представления:

- вершины графа разбиваются на непересекающиеся ярусы;
- решётка имеет единственную начальную и единственную конечную вершину;
- рёбра соединяют только вершины соседних ярусов;
- каждому ребру приписан символ кодового слова;

- каждый полный путь от начальной до конечной вершины соответствует некоторому кодовому слову;
- для каждого кодового слова существует хотя бы один соответствующий путь.

В такой интерпретации задача декодирования превращается в поиск оптимального пути на графе.

6 Декодирование с использованием решётки

Для двоично-симметричного канала и критерия минимума расстояния Хэмминга вес ребра выбирается следующим образом:

- если символ на ребре совпадает с соответствующим принятым символом, вес ребра равен 0;
- в противном случае вес ребра равен 1.

Вес пути вычисляется как сумма весов составляющих его рёбер. Если путь соответствует кодовому слову $\mathbf{c}$, то

$$\text{вес пути} = d_H(\mathbf{y}, \mathbf{c}).$$

Следовательно, задача декодирования сводится к поиску кратчайшего пути в решётке.

Алгоритм Витерби реализует эту идею следующим образом:

1. на каждом ярусе обрабатывается очередной символ принятого слова;
2. для каждого допустимого ребра вычисляется локальный вес;
3. к локальному весу добавляется накопленная метрика пути, ведущего в начальную вершину ребра;
4. для каждой вершины следующего яруса сохраняется только путь с минимальной накопленной метрикой.

Таким образом, на каждом шаге отбрасываются заведомо неоптимальные продолжения, что позволяет избежать полного перебора всех кодовых слов.

7 Профиль решётки и вычислительная сложность

Эффективность декодирования определяется не только длиной кода, но и структурой решётки. Для одного и того же кода можно построить различные решётки, существенно различающиеся по сложности.

Для характеристики сложности вводится понятие профиля решётки — вектора, указывающего число вершин на каждом ярусе. Например, профиль

$$(1, 2, 2, 1)$$

означает, что на первом ярусе имеется одна вершина, на втором — две, на третьем — две, на последнем — снова одна.

Чем меньше вершин содержит решётка, тем меньше состояний требуется одновременно поддерживать в процессе декодирования, что напрямую влияет на производительность.

8 Построение решётки для произвольного блочного кода

Рассмотрим код, состоящий из M кодовых слов длины n:

$$\mathbf{c}_m = (c_{m1}, c_{m2}, \ldots, c_{mn}), \quad m = 1, \ldots, M.$$

На i-м ярусе каждое кодовое слово разбивается на две части:

$$\mathbf{c}_m = (\mathbf{c}_m^P, \mathbf{c}_m^F),$$

где $\mathbf{c}_m^P$ — префикс длины i (прошлое), а $\mathbf{c}_m^F$ — суффикс длины $n - i$ (будущее).

Для фиксированного прошлого $\mathbf{c}^P$ определим множество возможных будущих:

$$\mathcal{F}_i(\mathbf{c}^P) = \{\mathbf{c}_m^F \mid \mathbf{c}_m = (\mathbf{c}^P, \mathbf{c}_m^F) \in \mathcal{C}\}.$$

Два различных префикса на ярусе i считаются эквивалентными, если соответствующие множества будущих совпадают. Такие префиксы отображаются в одну вершину решётки. Это ключевой механизм схлопывания:

- одинаковое множество будущих определяет одну и ту же вершину;
- одинаковое прошлое задаёт множество входящих рёбер.

9 Правило соединения вершин

Пусть имеется вершина на ярусе i, соответствующая некоторому классу прошлых фрагментов, и вершина на ярусе $i+1$, соответствующая более длинным префиксам. Ребро между ними проводится тогда и только тогда, когда существует кодовое слово, удовлетворяющее двум условиям:

1. его префикс длины $i+1$ принадлежит классу, представляемому вершиной следующего яруса;
2. после отбрасывания последнего символа этого префикса получается префикс длины i, принадлежащий классу, представляемому вершиной текущего яруса.

Иными словами, ребро существует, если прошлое следующего яруса получается из прошлого текущего яруса добавлением одного символа. Метка ребра — это добавленный символ.

10 Начальные этапы построения решётки

Рассмотрим пример кода из 8 кодовых слов длины 6. Решётка в этом случае будет содержать 7 ярусов.

Нулевой ярус:

item единственная вершина;

- прошлое пусто;
- будущее состоит из всех кодовых слов;
- входящие рёбра отсутствуют.

Первый ярус:

- возможные префиксы длины 1: 0 и 1;
- для каждого префикса строится множество допустимых будущих;

- различные множества будущих порождают различные вершины;
- из начальной вершины выходят рёбра с метками 0 и 1.

Второй ярус:

- рассматриваются префиксы длины 2: 00, 01, 10, 11;
- для каждого префикса вновь определяется множество будущих;
- вершины и рёбра определяются по тому же принципу.

Процедура повторяется вплоть до последнего яруса. Для практической реализации удобно вести таблицу, фиксирующую для каждого яруса: номер яруса, рассматриваемый префикс, множество будущих, вершину предыдущего яруса, вершину текущего яруса и метку ребра.

11 Значение решётчатого представления

Решётка предоставляет:

- компактное описание множества кодовых слов;
- возможность пошагового вычисления метрики пути;
- основу для эффективного декодирования методом динамического программирования (алгоритм Витерби).

В контексте двоично-симметричного канала:

- каждое кодовое слово представляется путём в решётке;
- каждое несовпадение с принятым символом увеличивает вес пути на единицу;
- оптимальный путь соответствует минимальному суммарному расстоянию Хэмминга;
- найденное таким образом кодовое слово совпадает с результатом декодирования по максимуму правдоподобия.